

Chapter 5 explain

شرح السلايدات 1 - 15

CHAPTER 5: The Relational Data Model and Relational Database Constraints

الفصل 5: النموذج العلاقي للبيانات وقواعد البيانات العالقية

هذا الفصل يشكل حجر الأساس في فهم نظم قواعد البيانات العالقية (Relational Databases)، وهو النموذج الأكثر استخدامًا في العالم اليوم. تم اقتراحه من قبل **E.F. Codd** في عام 1970، وغيّر طريقة تخزين واسترجاع البيانات بشكل جذري.

Relational Model Concepts

مفاهيم النموذج العلاقي

النموذج العلاقي (Relational Model) هو نموذج رياضي لتمثيل البيانات باستخدام المفاهيم من نظرية المجموعات (Set Theory).

- **العلاقة (Relation)** هي الوحدة الأساسية في النموذج العلاقي.
- يمكن التفكير في العلاقة على أنها جدول ثنائي الأبعاد (صفوف وأعمدة).
- كل علاقة تمثل كيانًا (Entity) أو علاقة (Relationship) في العالم الحقيقي.
- مثال: جدول STUDENT يمثل طلابًا في الجامعة.

ملاحظة مهمة:

النموذج العلاقي ليس مجرد جداول، بل له أساس رياضي قوي مبني على نظرية المجموعات، وهذا ما يجعل العمليات عليه دقيقة وقابلة للتحليل.

The Relational Model of Data is based on the concept of a Relation

النموذج العلاقي للبيانات مبني على مفهوم العلاقة

- العلاقة ليست مجرد جدول عادي، بل هي مجموعة من الصفوف (tuples) بدون ترتيب، ولا تحتوي على تكرار.
- تعتمد على مفاهيم من المنطق الرياضي ونظرية المجموعات.
- كل صف في الجدول يمثل معلومة واحدة كاملة عن كيان ما.

ملاحظة:

العلاقة = مجموعة من الصفوف (tuples)، والصفوف غير مرتبة، ولا يمكن تكرار أي صفين متماثلين.

A Relation is a mathematical concept based on the ideas of sets

العلاقة مفهوم رياضي مبني على فكرة المجموعات

- مثل أي مجموعة في الرياضيات:
- لا يوجد ترتيب للعناصر (الصفوف).
- لا توجد عناصر مكررة.
- هذا يعني أن:
- لا يمكن أن يكون هناك صفان متماثلان تمامًا في العلاقة.
- ترتيب الصفوف لا يهم (مثلاً: الصف الأول أو الثاني ليس له معنى منطقي).

ملاحظة:

في الواقع العملي (مثل SQL)، قد تُسمح الصفوف المكررة، لكن في النموذج العلاقي الصافي، يُمنع التكرار.

The model was first proposed by Dr. E.F. Codd of IBM Research in 1970

تم اقتراح النموذج لأول مرة من قبل الدكتور E.F. Codd من بحوث IBM في عام 1970

- الورقة البحثية:
- "A Relational Model for Large Shared Data Banks" – يونيو 1970 – منشور في مجلة ACM.
- هذه الورقة أحدثت ثورة في عالم قواعد البيانات.
- نتيجة لذلك، حصل Codd على جائزة تورينغ (ACM Turing Award)، وهي أعلى جائزة في علوم الحاسوب.

ملاحظة مهمة للامتحان:

هو "أبو" النموذج العلاقي، وتاريخ 1970 واسم الورقة والمجلة من المعلومات الأساسية التي قد تُسأل عنها Codd.

Informal Definitions

التعريفات غير الرسمية

قبل الدخول في المصطلحات الرسمية، نبدأ بتعريفات بسيطة تشبه الجداول التي نراها في Excel.

- العلاقة (Relation) ← تشبه جدولاً.
- الصف (Row) ← يمثل معلومة واحدة عن كيان (مثل طالب).

- **العمود (Column)** ← له عنوان (مثل: الاسم، الرقم الوطني).
- **الصفوف** تمثل **حقائق** عن كيانات أو علاقات في العالم الحقيقي.
- في النموذج الرسمي:
- الصف → يسمى **n-tuple** أو **tuple** (صيغة الجمع: tuples).
- عنوان العمود → يسمى **Attribute** (سمة).

📌 ملاحظة:

"tuple" لكن في النظرية نقول "row" نقول SQL هو مصطلح رياضي، ويعني مجموعة مرتبة من القيم. في "Tuple".

Example of a Relation

مثال على علاقة

(الشريحة 5-6 تحتوي على مثال بصري لجدول STUDENT، لكنه غير موجود نصيًا هنا، لكننا نفهم من السياق)

- مثال: جدول STUDENT به الأعمدة:
 - Name, SSN, Department, Age, etc.
 - كل صف يمثل طالبًا واحدًا.
 - كل عمود يمثل خاصية للطالب.

📌 ملاحظة:

حتى لو لم ترَ الجدول، فالفكرة أن العلاقة = جدول، الصف = طالب، العمود = خاصية.

Key of a Relation

المفتاح في العلاقة

- كل صف في الجدول يجب أن يكون فريدًا.
- **المفتاح (Key)** هو قيمة (أو مجموعة قيم) تُستخدم لتحديد الصف فريدًا.
- مثال: في جدول STUDENT، يكون رقم الضمان الاجتماعي (SSN) هو المفتاح لأنه فريد لكل طالب.
- أحيانًا لا يوجد حقل طبيعي فريد، فيُضاف حقل اصطناعي مثل:
 - ID (1, 2, 3, ...).
 - يُسمى هذا مفتاح اصطناعي (Artificial Key) أو مفتاح بديل (Surrogate Key).

📌 ملاحظات مهمة:

- المفتاح الطبيعي (Natural Key): مثل SSN أو البريد الإلكتروني.
- المفتاح الاصطناعي (Surrogate Key): مثل StudentID، لا معنى له خارج قاعدة البيانات، لكنه يضمن التفرد.
- وجود المفتاح ضروري لضمان التمييز بين الصفوف.

Formal Definitions - Schema

التعريفات الرسمية – المخطط (Schema)

- **مخطط العلاقة (Relation Schema)** هو وصف للعلاقة، وليس بياناتها.
- يُكتب بالشكل:

$R(A_1, A_2, \dots, A_n)$

- R : اسم العلاقة (مثل: STUDENT).
- $A_1, A_2, \dots, A_n$: للأسماء (السمات) للأعمدة (مثل: SSN, Name, Age).

• مثال:

$CUSTOMER(Cust-id, Cust-name, Address, Phone\#)$

- اسم العلاقة: $CUSTOMER$.
- السمات: $Cust-id, Cust-name$, إلخ.
- كل سمة (Attribute) لها نطاق (Domain): مجموعة القيم المسموحة لها.
- مثال: نطاق $Cust-id$ = أرقام مكونة من 6 أرقام.
- نطاق $\#Phone$ = أرقام هواتف بتنسيق معين.

♦ ملاحظة:

المخطط هو "الهيكِل"، مثل تصميم الجدول، ولا يتغير غالبًا.

Formal Definitions - Tuple

التعريفات الرسمية – البُعد (Tuple)

- الـ **tuple** هو صف واحد في العلاقة.
- هو مجموعة مرتبة من القيم، تُكتب بين زاويتين: $\langle \dots \rangle$.
- كل قيمة في الـ tuple تعود إلى النطاق (domain) الخاص بالسمة.
- مثال:

$\langle 632895, "John Smith", "101 Main St. Atlanta, GA 30332", "(404) 894-2000 \rangle$

- هذا tuple له 4 قيم، يُسمى **tuple-4**.
- كل قيمة تتوافق مع سمة في $CUSTOMER$.
- العلاقة نفسها هي مجموعة من الـ **tuples**.

♦ ملاحظات:

- الترتيب مهم: القيمة الأولى دائماً لـ $Cust-id$ ، الثانية لـ $Cust-name$ ، إلخ.
- لا يمكن أن يكون هناك تكرار في الـ tuples داخل نفس العلاقة (لأنها مجموعة رياضية).

Formal Definitions - Domain

التعريفات الرسمية – النطاق (Domain)

- **النطاق (Domain)** هو مجموعة القيم الممكنة لسمة معينة.
- له جانبان:
- 1. **تعريف منطقي (Logical Definition):** ماذا تمثل القيم؟
 - مثال: "USA_phone_numbers" = مجموعة أرقام الهواتف الأمريكية الصالحة.
- 2. **صيغة أو نوع بيانات (Format/Data Type):**
 - مثال: ddd-dddd(ddd) ← كل d رقم عشري.
- **السمة (Attribute)** تُعرف على نطاق، لكنها تُعطي معنى للنطاق في سياق العلاقة.
- مثال: النطاق "Date" يمكن استخدامه في سمتين:
- Invoice-date (تاريخ الفاتورة)
- Payment-date (تاريخ الدفع)
- نفس النوع، لكن المعنى مختلف.

♦ ملاحظة مهمة:

النطاق يحدد "ما يمكن أن تكون عليه القيمة"، والسمة تحدد "ماذا تعني هذه القيمة".

Formal Definitions - State

التعريفات الرسمية – الحالة (State)

- **حالة العلاقة (Relation State)** هي مجموعة الصفوف الفعلية في العلاقة عند لحظة معينة.
- رياضياً:
- $$r(R) \subseteq \text{dom}(A1) \times \text{dom}(A2) \times \dots \times \text{dom}(An)$$
- أي: الحالة هي مجموعة جزئية من ضرب الديكارتي لنطاقات السمات.
- مثال:
- $\text{dom}(\text{Cust-name}) = \text{varchar}(25)$ ← نص بطول 25 حرف كحد أقصى.
- لكن ليس كل النصوص في هذا النطاق موجودة في الجدول.
- الحالة $r(R)$ تحتوي فقط على الصفوف المدخلة فعلياً.

♦ ملاحظة:

- المخطط (Schema): ثابت (مثل تصميم الجدول).
- الحالة (State): متغيرة (تحديث، إدخال، حذف).

Formal Definitions - Summary

التعريفات الرسمية – ملخص

الصيغة العامة:

- علاقة بـ n سمة: $R(A1, A2, \dots, An)$
- الحالة: $r(R) \subset \text{dom}(A1) \times \text{dom}(A2) \times \dots \times \text{dom}(An)$

- $r(R) = \{t_1, t_2, \dots, t_k\} \leftarrow$ مجموعة من ال tuples.
- كل $\langle \text{tuple}: t_i = \langle v_1, v_2, \dots, v_n \rangle$ حيث $v_j \in \text{dom}(A_j)$

تفسير رياضي بسيط:

- الضرب الديكارتي: كل التوافق الممكنة للقيم.
- الحالة: فقط التوافق المستخدمة فعليًا.

Formal Definitions - Example

التعريفات الرسمية – مثال تطبيقي

- العلاقة: $R(A_1, A_2)$
- $\text{dom}(A_1) = \{0, 1\}$
- $\text{dom}(A_2) = \{a, b, c\}$
- الضرب الديكارتي:
- $\{ \langle a \rangle, \langle 0, b \rangle, \langle 0, c \rangle, \langle 1, a \rangle, \langle 1, b \rangle, \langle 1, c \rangle, \langle 0 \rangle \} \leftarrow$ 6 توافق ممكنة.
- الحالة $r(R)$ يمكن أن تكون أي مجموعة جزئية من هذه.
- مثال: 3 $\leftarrow r(R) = \{ \langle 0, a \rangle, \langle 0, b \rangle, \langle 1, c \rangle \}$ صفوف فقط.
- هذه الحالة تمثل "تمثيل فعلي" للعلاقة.

ملاحظة:

العلاقة لا يجب أن تحتوي كل التوافق الممكنة، بل فقط ما تم إدخاله.

Definition Summary (Table of Terms)

ملخص المصطلحات (جدول المقابلة)

Informal Terms	Formal Terms
Table	Relation
Column Header	Attribute
All possible Column Values	Domain
Row	Tuple
Table Definition	Schema of a Relation
Populated Table	State of the Relation

الترجمة:

المصطلح غير الرسمي	المصطلح الرسمي
جدول	علاقة (Relation)
عنوان العمود	سمة (Attribute)
جميع القيم الممكنة للعمود	نطاق (Domain)
صف	بُعد (Tuple)
تعريف الجدول	مخطط العلاقة (Schema)
جدول معبأ بالبيانات	حالة العلاقة (State)

📌 ملاحظة للامتحان:

هذا الجدول مهم جداً. تأكد من حفظ كل مقابلة. قد يُطلب منك تحويل مصطلح من غير رسمي إلى رسمي.

Example – A relation STUDENT

مثال – علاقة STUDENT

(الشريحة 5-15 تشير إلى وجود مثال لعلاقة STUDENT، لكن التفاصيل غير مكتوبة في النص المتوفر)

لكن من السياق، يمكننا استنتاج أن:

- العلاقة: STUDENT(SSN, Name, Department, Age, GPA)
- المخطط: STUDENT(SSN, Name, Department, Age, GPA)
- الحالة: تحتوي على صفوف مثل:
 - <123-45-6789, "Ali", "CS", 20, 3.7>
 - <987-65-4321, "Sara", "Math", 19, 3.9>
- المفتاح: SSN
- النطاقات:
 - SSN : أرقام بتنسيق XXX-XX-XXXX
 - Name : نص (varchar(50))
 - Age : عدد صحيح بين 16 و 100
 - GPA : رقم عشري بين 0.0 و 4.0

📌 ملاحظة:

هذا المثال التخيلي يوضح كيف تُطبّق كل المفاهيم السابقة على كيان حقيقي.

**خلاصة الفصل 5



المفهوم	الشرح
العلاقة (Relation)	جدول ذو صفوف وأعمدة، مبني على نظرية المجموعات
السمة (Attribute)	عمود في الجدول، له اسم ونطاق
التُبد (Tuple)	صف في الجدول، مجموعة مرتبة من القيم
النطاق (Domain)	مجموعة القيم الممكنة لسمة ما
المخطط (Schema)	هيكل العلاقة: الاسم + السمات + النطاقات
الحالة (State)	البيانات الفعلية (الصفوف) في وقت معين
المفتاح (Key)	سمة (أو مجموعة) تحدد الصف فريدًا
المفتاح الاصطناعي	مفتاح يُضاف (مثل ID) عندما لا يوجد مفتاح طبيعي
Codd (1970)	اقترح النموذج في ورقة شهيرة، حصل على جائزة تورينغ

شرح السلايدات 16 - 30

Characteristics Of Relations

خصائص العلاقات

هذه الشريحة توضح الخصائص الأساسية التي تميز العلاقة (Relation) في النموذج العلاقي.

Ordering of tuples in a relation $r(R)$:

ترتيب الصفوف (tuples) في العلاقة $r(R)$:

- الصفوف (tuples) في العلاقة لا تُعتبر مرتبة.
- حتى لو ظهرت في شكل جدول بترتيب معين (من الأعلى إلى الأسفل)، فإن هذا الترتيب ليس له معنى منطقي.
- أي: يمكن تبديل ترتيب الصفوف، وستظل العلاقة نفسها.
- السبب: العلاقة هي مجموعة رياضية، والمجموعة لا تعتمد على الترتيب.

🔴 ملاحظة مهمة:

إذا غيرت ترتيب الصفوف في الجدول، فإن العلاقة لا تتغير، لأن المجموعة $\{A, B\} = \{B, A\}$.

Ordering of attributes in a relation schema R (and of values within each tuple):

ترتيب السمات في مخطط العلاقة R (وقيم كل صف):

- على عكس الصفوف، فإن السمات (attributes) في المخطط $R(A_1, A_2, \dots, A_n)$ تُعتبر مرتبة.
- بنفس الطريقة، القيم في كل $\text{tuple } t = \langle v_1, v_2, \dots, v_n \rangle$ تكون مرتبة حسب تسلسل السمات.
- أي: v_1 تنتمي إلى A_1 ، v_2 إلى A_2 ، وهكذا.
- هذا الترتيب ضروري لربط كل قيمة بالسمة الصحيحة.

ملاحظة:

- هذا الترتيب ضروري في النموذج التقليدي.
- لكن هناك تعريفاً أكثر تعميماً (يُذكر في الشريحة) يسمح بتمثيل tuple كمجموعة من أزواج (اسم السمة، القيمة)، مثل:
 - $t = \{\langle \text{Name}, "John" \rangle, \langle \text{SSN}, 123456789 \rangle\}$
- هذا يُسمى تمثيلاً ذاتي الوصف (Self-describing) لأنه لا يعتمد على الترتيب.

ملاحظة للامتحان:

- ترتيب الصفوف: غير مهم (لا معنى له).
- ترتيب السمات والقيم: مهم (يحدد معنى كل قيمة).

Same state as previous Figure (but with different order of tuples)

نفس الحالة السابقة (ولكن بترتيب مختلف للصفوف)

- هذه الشريحة توضح أن تغيير ترتيب الصفوف لا يغير حالة العلاقة.
- تُظهر مثالاً على علاقة STUDENT ، حيث تم تبديل ترتيب الصفوف.
- الحالة (State) تظل نفسها لأن:
 - نفس الصفوف موجودة.
 - لا يوجد تكرار.
 - لا يهم الترتيب.

ملاحظة:

هذه الشريحة تؤكد الفكرة: العلاقة = مجموعة غير مرتبة من الصفوف.

Characteristics Of Relations

خصائص العلاقات (تتمة)

Values in a tuple:

القيم في البُعد (tuple):

- كل القيم في tuple يجب أن تكون ذرية (Atomic)، أي:
 - غير قابلة للانقسام.
 - لا يمكن أن تكون قائمة أو مجموعة.
- مثال: لا يمكن أن يكون `Phone# = {055555, 054444}` في عمود واحد (في النموذج العلاقي الصافي).
- كل قيمة v_i في `<tuple t = <v1, v2, ..., vn` يجب أن تكون من نطاق السمة `(dom(Ai))`.
- أي: يجب أن تكون من القيم المسموحة لتلك السمة.

Special Value: NULL

القيمة الخاصة: NULL

- تُستخدم للدلالة على:
 1. قيمة غير معروفة (Unknown)
 - مثال: عمر طالب لم يتم تسجيله بعد.
 2. قيمة غير متوفرة (Unavailable)
 - مثال: رقم الهاتف لم يُعطَ بعد.
 3. قيمة غير منطبقة (Inapplicable)
 - مثال: "تاريخ الزواج" لطالب لا يزال طالبًا.

ملاحظات مهمة: 📌

- ليس عددًا، ولا نصًا فارغًا، ولا صفرًا NULL.
- "يعني" لا نعرف" أو "لا ينطبق NULL".
- بعض السمات قد تمنع NULL (مثل: المفتاح الأساسي).

Notation

الرموز والترميز

هذه الشريحة توضح الطريقة الرسمية للإشارة إلى قيم السمات في tuple.

- نستخدم الترميز التالي للوصول إلى قيمة سمة معينة في `tuple t`:
 - A_i للسمة v_i تُعطي القيمة $t.A_i$ أو $t[A_i]$.
 - مثال: إذا كان `<t = <123, "Ali", 20`، و `A1=ID`، فإن `t[A1] = 123`.
 - يمكن الإشارة إلى مجموعة جزئية من السمات:
 - يحتوي قيم هذه السمات فقط subtree يُعطي `t[Au, Av, ..., Aw]`.
 - مثال: `<t[Name, Age] = <"Ali", 20`.

ملاحظة: 📌

هذا الترميز مهم جدًا في العمليات الجبرية على العلاقات (مثل الانتقاء والمشروع).

CONSTRAINTS

القيود (Constraints)

القيود هي شروط تُفرض على قاعدة البيانات لضمان صحة البيانات واتساقها.

أنواع القيود (Three Main Types):

النوع	الترجمة	الشرح
Inherent or Implicit Constraints	قيود ضمنية أو طبيعية	مبنية على النموذج نفسه. لا يمكن كسرها لأن النظام لا يسمح بها. مثال: لا يمكن أن تكون قيمة سمة قائمة (مثل: قائمة أرقام هواتف في عمود واحد) في النموذج العلاقي.
Schema-based or Explicit Constraints	قيود صريحة أو مبنية على المخطط	تُعرّف في مخطط قاعدة البيانات (Schema). مثال: المفتاح الأساسي، قيود النطاق، قيود الرجوع.
Application-based or Semantic Constraints	قيود تطبيقية أو دلالية	لا يمكن التعبير عنها في النموذج، ويجب فرضها ببرامج التطبيق. مثال: "درجة الطالب لا يمكن أن تزيد عن 100"، أو "الموظف لا يمكن أن يكون مديرًا لقسم إلا إذا كان عمره فوق 30".

ملاحظة للمتحان:

- القيود الضمنية: مفروضة من قبل النموذج.
- القيود الصريحة: تُكتب في المخطط (مثل SQL).
- القيود التطبيقية: تُنفذ بالكود (مثل Java, Python).

Relational Integrity Constraints

قيود سلامة البيانات العلاقية

هذه القيود ضرورية لضمان أن قاعدة البيانات تكون صالحة وثابتة.

أنواع القيود الصريحة (Explicit Schema-Based Constraints):

- Key Constraints – قيود المفتاح
- Entity Integrity Constraints – قيود سلامة الكيان
- Referential Integrity Constraints – قيود السلامة المرجعية
- Domain Constraints – قيود النطاق (مذكورة كملاحظة)

Domain Constraint

- كل قيمة في tuple يجب أن تكون من نطاق السمة (domain).
- أو تكون NULL إذا كان مسموحًا بها لتلك السمة.

مثال: 

إذا كان نطاق Age هو أعداد صحيحة من 1 إلى 150، فإن Age = 200 أو Age = "abc" يُعد انتهاكًا لقيود النطاق.

Key Constraints

قيود المفتاح

هذه القيود تضمن أن كل صف في الجدول يمكن تمييزه بشكل فريد.

Superkey of R:

المفتاح الفائق (Superkey)

- هو مجموعة من سمة (أو سمة واحدة) بحيث لا يمكن لصفيين أن يكون لهما نفس القيمة في هذه السمات.
- بمعنى: القيمة في هذه السمات فريدة لكل tuple.
- الشرط الرياضي:
- لكل صفين مختلفين t_1 و t_2 في العلاقة $r(R)$:

$$t_1[SK] \neq t_2[SK]$$

حيث SK هو المفتاح الفائق.

مثال: 

في جدول CAR(State, Reg#, SerialNo, Make, Model, Year)

- {State, Reg#} ← مفتاح فائق (رقم التسجيل مرتبط بالولاية)
- {SerialNo} ← مفتاح فائق (رقم تسلسلي فريد عالميًا)
- {SerialNo, Make} ← أيضًا مفتاح فائق، لكنه ليس أصغر ممكن

Key of R:

المفتاح (Key)

- هو مفتاح فائق أصغري (Minimal Superkey).
- أي: إذا أزلت أي سمة منه، يفقد خاصية التفرد.
- لا يمكن تقليله أكثر دون فقدان التفرد.

- كل مفتاح هو مفتاح فائق، لكن العكس غير صحيح.
- يمكن أن يكون للعلاقة عدة مفاتيح (تسمى مفاتيح مرشحة Candidate Keys).

Key Constraints (continued)

قيود المفاتيح (تتمة)

مثال: CAR Relation

- `CAR(State, Reg#, SerialNo, Make, Model, Year)`
- المفاتيح المرشحة (Candidate Keys):
 1. `{State, Reg#}` ← لأن رقم التسجيل ليس فريدًا عالميًا، بل فريد داخل الولاية
 2. `{SerialNo}` ← لأنه فريد عالميًا
- إذا:
 - كلا المجموعتين هما مفتاحان ومفتاحان فائقان.
 - (ويبقى التفرد Make يمكن إزالة) مفتاح فائق، لكن ليس مفتاحًا لأنه ليس أصغري ← `{SerialNo, Make}`

📌 قواعد عامة:

- أي مفتاح هو مفتاح فائق.
- أي مجموعة تحتوي على مفتاح هي مفتاح فائق.
- أصغر مفتاح فائق هو مفتاح.

Key Constraints (continued)

قيود المفاتيح (تتمة)

Primary Key (المفتاح الأساسي)

- إذا كانت هناك عدة مفاتيح مرشحة، نختار واحدًا منها ليكون المفتاح الأساسي (Primary Key).
- يُختار بشكل تعسفي، لكن غالبًا:
 - الأصغر حجمًا (أقل عدد سمات).
 - الأسهل في الاستخدام.
 - في المخطط، يُوضع خط تحته لتمييزه.
- مثال: `CAR(State, Reg#, **SerialNo**, Make, Model, Year)`

أهمية المفتاح الأساسي:

1. تحديد هوية (Tuple Identity).

2. الرجوع إلى هذا tuple من علاقات أخرى (Referencing).
• مثال: جدول OWNER قد يحتوي على SerialNo كمرجع لسيارة.

🔴 ملاحظة مهمة:

- المفتاح الأساسي لا يمكن أن يكون NULL.
- يجب أن يكون فريدًا وحقيقيًا (غير قابل للتغيير غالبًا).

CAR table with two candidate keys – LicenseNumber chosen as Primary Key

جدول CAR مع مفتاحين مرشحين – تم اختيار LicenseNumber كمفتاح أساسي

هذه الشريحة توضح مثالًا بصريًا (غير متوفر نصيًا) لجدول سيارات به:

- مفتاحين مرشحين: LicenseNumber و SerialNo.
- تم اختيار LicenseNumber كمفتاح أساسي (مُسَطَّر تحته).

🔴 ملاحظة:

حتى لو كان SerialNo فريدًا عالميًا، يمكن اختيار LicenseNumber كمفتاح أساسي إذا كان أكثر استخدامًا في النظام.

Relational Database Schema

مخطط قاعدة البيانات العلاقية

هذا المفهوم يوسع الفكرة من علاقة واحدة إلى قاعدة بيانات كاملة.

التعريف:

- مخطط قاعدة البيانات العلاقية (S) هو:
 $S = \{R1, R2, \dots, Rn\}$ مجموعة من مخططات العلاقات.
- بالإضافة إلى مجموعة من قيود السلامة (IC).
- كل R_i هو مخطط علاقة (مثل: STUDENT, DEPARTMENT, COURSE).
- هو اسم قاعدة البيانات ككل S.

🔴 مثال:

مخطط قاعدة بيانات COMPANY يحتوي على 6 علاقات:

- EMPLOYEE , DEPARTMENT , DEPT_LOCATIONS , PROJECT , WORKS_ON , DEPENDENT

🔴 ملاحظة:

المخطط هو "التصميم"، ولا يتغير كثيرًا.

COMPANY Database Schema

مخطط قاعدة بيانات COMPANY

(الشريحة تحتوي على رسم بياني لمخطط COMPANY، غير متوفر نصيًا)

لكن من السياق، نفهم أن:

- يحتوي على 6 جداول (علاقات).
- كل جدول له مفتاح أساسي.
- هناك روابط بين الجداول (مثل: Dno في EMPLOYEE يشير إلى Dnumber في DEPARTMENT).

ملاحظة:

هذا المخطط سيكون مثالاً أساسياً في الفصول القادمة (مثل الاستعلامات والعمليات).

Relational Database State

حالة قاعدة البيانات العلاقية

حالة قاعدة البيانات هي البيانات الفعلية المخزنة في كل العلاقات عند لحظة معينة.

التعريف:

- حالة قاعدة البيانات (DB) هي:
- مجموعة من حالات العلاقات: $DB = \{r_1, r_2, \dots, r_m\}$
- بحيث r_i هي حالة R_i .
- وتخضع جميعها للقيود المحددة في IC.
- تُسمى أحياناً **لقطة (Snapshot)** أو **مثيل (Instance)**.
- لكن نتجنب كلمة "Instance" لأنها قد تشير إلى tuple واحد.

الحالة الصالحة (Valid State):

- حالة تحقق جميع القيود.
- إذا انتهكت قيداً واحداً، فهي حالة غير صالحة (Invalid State).

ملاحظة:

- المخطط: ثابت.
- الحالة: متغيرة (تتغير بالـ INSERT, UPDATE, DELETE).

Populated database state

هذه الشريحة توضح أن كل علاقة تحتوي على عدة صفوف (tuples) في حالتها الفعلية.

الخصائص:

- كل علاقة لديها حالة حالية (current state) تحتوي على صفوف كثيرة.
- حالة قاعدة البيانات الكلية هي اتحاد حالات جميع العلاقات.
- عند كل تغيير (إدخال، حذف، تعديل)، تنشأ حالة جديدة.

العمليات الأساسية (Basic Operations):

1. INSERT – جديد tuple إدخال.
2. DELETE – موجود tuple حذف.
3. MODIFY – موجود tuple تعديل قيمة سمة في.

ملاحظة مهمة:

هذه العمليات قد تنتهك القيود، لذا يجب التحقق منها قبل تنفيذها.

Populated database state for COMPANY

حالة معبأة لقاعدة بيانات COMPANY

(الشريحة تُظهر مثالاً عملياً على بيانات حقيقية في جداول COMPANY)

من المثال (الذي لا يمكن رؤيته نصياً)، نستنتج أن:

- كل جدول (مثل EMPLOYEE) يحتوي على صفوف فعلية.
- القيم تتوافق مع النطاقات.
- المفاتيح الأساسية فريدة.
- الروابط بين الجداول (مثل Dno) تشير إلى قيم موجودة.

ملاحظة للامتحان:

- تأكد من فهم كيفية قراءة الجداول.
- كيف تُستخدم المفاتيح للربط بين الجداول.

✓ خلاصة الفصل 5 – الجزء الثاني (الشريحة 16-30) – أهم النقاط

المفهوم	الشرح
ترتيب الصفوف	غير مهم (العلاقة مجموعة غير مرتبة)

المفهوم	الشرح
ترتيب السمات	مهم (يحدد معنى القيم)
القيم الذرية	كل قيمة غير قابلة للانقسام
NULL	يمثل: غير معروف، غير متوفر، غير منطبق
المفتاح الفائق (Superkey)	مجموعة سمات تضمن التفرد
المفتاح (Key)	مفتاح فائق أصغري
المفتاح الأساسي (Primary Key)	يُختار من المفاتيح المرشحة، لا يمكن أن يكون NULL
مخطط قاعدة البيانات	مجموعة مخططات + قيود
حالة قاعدة البيانات	البيانات الفعلية في جميع الجداول
الحالة الصالحة	تحقق جميع القيود
العمليات الأساسية	INSERT, DELETE, MODIFY

نصائح :

1. افهم الفرق بين:

- Schema vs State
- Superkey vs Key vs Primary Key
- Implicit vs Explicit vs Semantic Constraints

2. اكتب الأمثلة:

- أعط مثالاً على علاقة مع حالة.
- احسب المفاتيح الفائقة والمفاتيح.

3. اشرح لماذا لا يُسمح بـ NULL في المفتاح الأساسي؟

- لأنه يكسر التفرد والهوية.

4. ما معنى أن العلاقة "ذرية"؟

- لا تحتوي على قيم مركبة أو قوائم.

شرح السلايدات من 30 - 43

Entity Integrity

سلامة الكيان (Entity Integrity)

هذه الشريحة تشرح قيدًا أساسيًا يضمن أن كل كيان (مثل طالب، موظف) يمكن تمييزه بشكل فريد.

- **محتوى القيد:**
لا يمكن أن تحتوي سمات المفتاح الأساسي (Primary Key) في أي علاقة على القيمة NULL.
- **السبب:**
لأن المفتاح الأساسي يُستخدم لتحديد هوية كل صف (tuple) في الجدول. إذا كان NULL، فهذا يعني أننا لا نعرف من هو هذا الكيان، وبالتالي لا يمكن التمييز بينه وبين غيره.
- **الصيغة الرياضية:**
لكل tuple t في حالة العلاقة $r(R)$:

$$t[PK] \neq NULL$$

- حيث PK هو المفتاح الأساسي.
إذا كان المفتاح الأساسي مكوناً من أكثر من سمة (مفتاح مركب)، فإن أي سمة فيه لا يمكن أن تكون NULL.

مثال: 

في جدول EMPLOYEE(SSN, Name, Dept):

- إذا كان SSN هو المفتاح الأساسي، فلا يمكن أن يكون SSN = NULL لأي موظف.
- حتى لو كانت القيمة غير معروفة، لا يُسمح بـ NULL هنا.

ملاحظة مهمة للامتحان: 

- Entity Integrity يخص المفتاح الأساسي فقط.
- السمات الأخرى (غير المفتاح) يمكن أن تكون NULL، إلا إذا تم تقييدها بقيود أخرى.

Referential Integrity

السلامة المرجعية

هذه الشريحة تقدم مفهوماً حيويًا في قواعد البيانات العلاقة: الربط بين الجداول.

- **التعريف:**
قيد يربط جدولين (علاقتيين):
- **الجدول المرجع (Referencing Relation)** ← يحتوي على مفتاح أجنبي.
- **الجدول المُشار إليه (Referenced Relation)** ← يحتوي على المفتاح الأساسي.
- **الغرض:**
ضمان أن أي قيمة في المفتاح الأجنبي تُشير إلى قيمة موجودة فعلاً في المفتاح الأساسي للجدول الآخر.
- **مثال:**
جدول Dno ← EMPLOYEE(Dno) هو مفتاح أجنبي.
جدول Dnumber ← DEPARTMENT(Dnumber) هو المفتاح الأساسي.
إذًا: كل قيمة في EMPLOYEE.Dno يجب أن تكون موجودة في DEPARTMENT.Dnumber.

ملاحظة: 

- هذا القيد يمنع "الإشارات الميتة" (Dead References)، مثل موظف ينتمي إلى قسم غير موجود.

Referential Integrity (continued)

السلامة المرجعية (تتمة)

هذه الشريحة توضح الآلية الدقيقة للربط بين الجداول.

المفتاح الأجنبي (Foreign Key - FK):

- هو مجموعة من سمة في العلاقة R1 تُشير إلى المفتاح الأساسي في العلاقة R2.
- (Referencing Relation) تُسمى العلاقة المرجعية R1.
- (Referenced Relation) تُسمى العلاقة المُشار إليها R2.

الشرط:

لكل tuple t_1 في R1 ، يجب أن تكون قيمة $t_1[FK]$ مساوية لقيمة $t_2[PK]$ لـ tuple t_2 في R2 ، أو تكون NULL.

التمثيل البياني:

يتم رسم سهم (Arc) من المفتاح الأجنبي في R1 إلى الجدول R2 (أو إلى المفتاح الأساسي فيه).

ملاحظة:

- العلاقة المرجعية = الجدول الذي يحتوي على "الإشارة".
- العلاقة المُشار إليها = الجدول "الذي تُشير إليه".

Referential Integrity (or foreign key) Constraint

قيد السلامة المرجعية (أو قيد المفتاح الأجنبي)

هذه الشريحة توضح القاعدة الرسمية للسلامة المرجعية.

بيان القيد:

- القيمة في المفتاح الأجنبي (FK) في الجدول المرجع R1 يمكن أن تكون:
 1. قيمة موجودة في المفتاح الأساسي (PK) للجدول المُشار إليه R2.
 2. قيمة NULL.
- مثال: Dno = 5 في EMPLOYEE ، و Dnumber = 5 موجود في DEPARTMENT.
- مثال: موظف لا ينتمي إلى قسم بعد (في فترة التدريب).

استثناء مهم:

- إذا كان المفتاح الأجنبي جزءاً من المفتاح الأساسي في الجدول المرجع R1 ، فلا يمكن أن يكون NULL.
- لأن المفتاح الأساسي لا يمكن أن يحتوي على NULL (Entity Integrity).

ملاحظة للامتحان:

- مسموح في المفتاح الأجنبي بشرط ألا يكون جزءاً من المفتاح الأساسي NULL.

Displaying a relational database schema and its constraints

عرض مخطط قاعدة البيانات العلاقية وقيودها

هذه الشريحة تشرح كيفية تمثيل المخطط بيانيًا.

- طريقة العرض:
 - كل علاقة تُعرض كصف من أسماء السمات.
 - اسم العلاقة يُكتب فوق الأسماء.
 - المفتاح الأساسي يُوضع تحته خط (Underlined).
 - المفتاح الأجنبي يُربط بسهم (Arc) مع الجدول المُشار إليه.
- السهم (Arc):
 - يبدأ من المفتاح الأجنبي في الجدول المرجع.
 - ينتهي عند الجدول المُشار إليه (أو عند المفتاح الأساسي فيه).
- هذا التمثيل يُسمى مخطط قاعدة البيانات العلاقية (Relational Schema Diagram).

ملاحظة:

- هذا الشكل البياني هو الأساس في فهم العلاقات بين الجداول.
- سيُستخدم بكثرة في الفصول القادمة (مثل الجبر العلاقي، SQL).

Referential Integrity Constraints for COMPANY database

قيود السلامة المرجعية لقاعدة بيانات COMPANY

(الشريحة تُظهر رسمًا بيانيًا لقاعدة بيانات COMPANY، غير متوفر نصيًا)

لكن من السياق، يمكننا استنتاج القيود التالية:

- EMPLOYEE.Ssn → المفتاح الأساسي.
- EMPLOYEE.Dno → مفتاح أجنبي يشير إلى DEPARTMENT.Dnumber .
- DEPARTMENT.Mgr_ssn → مفتاح أجنبي يشير إلى EMPLOYEE.Ssn (المدير هو موظف).
- DEPT_LOCATIONS.Dnumber → مفتاح أجنبي يشير إلى DEPARTMENT.Dnumber .
- WORKS_ON.Essn → EMPLOYEE.Ssn يشير إلى .
- WORKS_ON.Pno → PROJECT.Pnumber يشير إلى .
- DEPENDENT.Essn → EMPLOYEE.Ssn يشير إلى .

ملاحظة مهمة:

- العلاقة بين EMPLOYEE و DEPARTMENT ثنائية الاتجاه:

- الموظف ينتمي إلى قسم.
- القسم له مدير (وهو موظف).
- هذه البنية تُظهر كيف تُبنى قواعد البيانات المعقدة من علاقات بسيطة.

Other Types of Constraints

أنواع أخرى من القيود

هذه الشريحة تتحدث عن القيود التي لا يمكن التعبير عنها في النموذج العلاقي مباشرة.

Semantic Integrity Constraints

قيود السلامة الدلالية

- هي قيود تعتمد على معنى التطبيق (Application Semantics).
- لا يمكن التعبير عنها باستخدام المفاتيح أو النطاقات أو الروابط.
- يجب فرضها ببرامج تطبيقية أو باستخدام أدوات متقدمة.

أمثلة:

- "الحد الأقصى لساعات العمل الأسبوعية لموظف ما هو 56 ساعة".
- "لا يمكن أن يكون مدير قسم أصغر من 30 سنة".
- "لا يمكن لطالب أن يسجل في أكثر من 6 مقررات في الفصل".

كيفية تنفيذها؟

- في SQL-99:

- لتعريف قيد عام: `CREATE ASSERTION`.
- `CREATE TRIGGER` (مثل `INSERT`) لتنفيذ إجراء عند حدوث حدث: `CREATE TRIGGER`.
- مثال: عند إدخال سجل في `WORKS_ON`، تحقق من أن مجموع الساعات لا يتجاوز 56.

ملاحظة:

- هذه القيود مهمة جدًا في الواقع العملي، لكنها خارج النموذج العلاقي الأساسي.
- غالبًا ما تُنسى في التصميم، مما يؤدي إلى بيانات غير صحيحة.

Update Operations on Relations

عمليات التحديث على العلاقات

هذه الشريحة تقدم العمليات الأساسية التي يمكن تنفيذها على قاعدة البيانات.

- العمليات الثلاث الأساسية:

1. **INSERT**: إدخال صف جديد.
2. **DELETE**: حذف صف موجود.
3. **MODIFY**: تعديل قيمة في صف موجود.

- الشرط الأساسي:

يجب ألا تؤدي هذه العمليات إلى انتهاك أي من قيود السلامة (Integrity Constraints).

- قد تحتاج عدة عمليات إلى التجميع معًا (Grouped) لضمان الاتساق.
- مثال: عند حذف قسم، يجب حذف جميع مواقع ومشاريعه.
- بعض التحديثات قد تنتشر تلقائيًا (Propagate) لصيانة الاتساق.
- مثال: عند تغيير رقم القسم، يتم تحديث جميع الموظفين الذين ينتمون إليه.

- ملاحظة:

- هذه العمليات هي الأساس في لغة التلاعب بالبيانات (DML) مثل (INSERT , DELETE , UPDATE) SQL.

Update Operations on Relations (continued)

عمليات التحديث على العلاقات (تتمة)

هذه الشريحة تشرح الإجراءات الممكنة عند انتهاك القيد.

عند انتهاك قيد نتيجة لعملية تحديث، يمكن اتخاذ أحد الإجراءات التالية:

1. إلغاء العملية (REJECT أو RESTRICT)

- رفض العملية تمامًا.
- مثال: لا يمكن حذف قسم له موظفون.

2. السماح بالعملية، ولكن إبلاغ المستخدم

- تنفيذ العملية مع إرسال تحذير.
- قد لا يكون مناسبًا دائمًا.

3. تنفيذ تحديثات إضافية تلقائيًا (CASCADE أو SET NULL)

- **CASCADE**: نقل التغيير تلقائيًا.

- مثال: عند حذف قسم، احذف جميع موظفيه.

- **SET NULL**: تعيين القيمة إلى NULL.

- مثال: عند حذف قسم، اجعل Dno = NULL لجميع موظفيه.

4. تشغيل برنامج تصحيح محدد من قبل المستخدم

- إجراء مخصص لمعالجة الحالة.

- ملاحظة مهمة:

- يجب تحديد استراتيجية التعامل مع الانتهاكات أثناء تصميم قاعدة البيانات.
- لا يمكن ترك القرار للنظام الافتراضي دائمًا.

Possible violations for each operation

الانتهاكات المحتملة لكل عملية

هذه الشريحة تحلل أي قيد يمكن انتهاكه حسب نوع العملية.

INSERT may violate any of the constraints:

عملية الإدخال قد تنتهك أي قيد:

1. Domain Constraint (قيد النطاق)

- إذا كانت قيمة السمة ليست من النطاق المحدد.
- مثال: إدخال "Age = "abc" أو Age = 200.

2. Key Constraint (قيد المفتاح)

- إذا كانت قيمة المفتاح (أو أحد المفاتيح المرشحة) مكررة.
- مثال: إدخال موظف برقم ضمان اجتماعي موجود مسبقاً.

3. Referential Integrity (السلامة المرجعية)

- إذا كانت قيمة المفتاح الأجنبي لا تشير إلى قيمة موجودة في الجدول المُشار إليه.
- مثال: إدخال موظف بـ Dno = 99 ، بينما لا يوجد قسم رقم 99.

4. Entity Integrity (سلامة الكيان)

- إذا كانت قيمة المفتاح الأساسي NULL .
- مثال: إدخال موظف بدون SSN .

ملاحظة: 📌

- عملية INSERT هي الأكثر خطورة لأنها قد تنتهك جميع القيود دفعة واحدة.

Possible violations for each operation (continued)

الانتهاكات المحتملة لكل عملية (تتمة)

DELETE may violate only referential integrity:

عملية الحذف قد تنتهك السلامة المرجعية فقط:

- السبب:
إذا كان المفتاح الأساسي للصف الذي يتم حذفه مُشار إليه من جدول آخر (عبر مفتاح أجنبي).
- مثال:
حذف قسم له موظفون.

- الموظفون ما زالوا يحتفظون بـ Dno يشير إلى القسم المحذوف → انتهاك للسلامة المرجعية.

طرق المعالجة (يجب تحديدها عند التصميم):

1. RESTRICT (الرفض)

- لا تسمح بالحذف إذا كان هناك مراجع.
- مثال: لا يمكن حذف قسم له موظفون.

2. CASCADE (الانتشار)

- احذف جميع الصفوف المرتبطة تلقائيًا.
- مثال: عند حذف قسم، احذف جميع موظفيه.

3. SET NULL (NULL تعيين القيمة إلى)

- اجعل المفتاح الأجنبي NULL في الجداول المرجعية.
- مثال: عند حذف قسم، اجعل Dno = NULL لجميع موظفيه.

ملاحظة: 

- لا يمكن أن تنتهك عملية DELETE قيد المفتاح أو النطاق، لأنها تُزيل بيانات، لا تضيفها.
- لكنها تهدد الاتساق المرجعي.

Possible violations for each operation (continued)

الانتهاكات المحتملة لكل عملية (تتمة)

UPDATE may violate multiple constraints:

عملية التعديل قد تنتهك عدة قيود:

1. Modifying a regular attribute (غير مفتاح):

- يمكن أن تنتهك:
- Domain Constraint: إذا كانت القيمة الجديدة غير صالحة.
- NOT NULL Constraint: NULL والقيمة الجديدة NULL إذا كانت السمة لا تقبل.

2. Updating the Primary Key (PK):

- مشابهة لـ DELETE ثم INSERT.
- يمكن أن تنتهك:
- Entity Integrity: NULL إذا أصبحت.
- Key Constraint: إذا كانت القيمة الجديدة مكررة.
- Referential Integrity: إذا كانت القيمة القديمة مُشار إليها من جدول آخر.

- الحلول: RESTRICT , CASCADE , SET NULL .

3. Updating a Foreign Key (FK):

- يمكن أن تنتهك:

- **Referential Integrity:** إذا كانت القيمة الجديدة لا تشير إلى صف موجود.
- **Entity Integrity:** NULL إذا كانت جزءاً من المفتاح الأساسي وأصبحت .

4. Updating a regular attribute (neither PK nor FK):

- يمكن أن تنتهك فقط:

- **Domain Constraint**
- **NOT NULL Constraint**

📌 ملاحظة:

- UPDATE هو الأكثر تعقيداً لأنه يؤثر على القيمة القديمة والجديدة.
- يجب التحقق من التأثيرات الجانبية.

Summary

ملخص

هذه الشريحة تلخص محتوى الفصل كاملاً.

- تم عرض مفاهيم النموذج العلاقي:
 - التعريفات الرسمية: العلاقة، السمة، البُعد، النطاق، المخطط، الحالة.
 - الخصائص: عدم ترتيب الصفوف، الذرية، استخدام NULL .
- تم مناقشة قيود النموذج العلاقي:
 1. قيود النطاق (Domain Constraints) – القيم من النطاق الصحيح.
 2. قيود المفتاح (Key Constraints) – التفرّد.
 3. سلامة الكيان (Entity Integrity) – المفتاح الأساسي لا يمكن أن يكون NULL .
 4. السلامة المرجعية (Referential Integrity) – المفتاح الأجنبي يشير إلى قيمة موجودة.
- تم وصف عمليات التحديث وكيفية التعامل مع انتهاكات القيود:
 - INSERT , DELETE , MODIFY .
 - استراتيجيات المعالجة: RESTRICT , CASCADE , SET NULL , TRIGGER .

✓ خلاصة شاملة – الفصل 5 كاملاً (من الشريحة 1 إلى 43)

المفهوم	الشرح المختصر
العلاقة (Relation)	جدول ذري، غير مرتب الصفوف، قيمه فردية.
المفتاح الأساسي (PK)	فريد، لا يمكن أن يكون NULL .
المفتاح الأجنبي (FK)	يشير إلى PK في جدول آخر، يمكن أن يكون NULL إلا إذا كان جزءاً من PK.
Entity Integrity	PK ≠ NULL.
Referential Integrity	FK = NULL أو (PK قيمة في).

المفهوم	الشرح المختصر
INSERT	قد ينتهك: Domain, Key, Referential, Entity.
DELETE	قد ينتهك: Referential فقط.
UPDATE	قد ينتهك: أي قيد حسب السمة المعدلة.
القيود الدلالية	تُنفذ ببرامج أو Triggers.
